TFG · SONDA PASIVA DE DETECCIÓN PQC

Prueba del TFG de Roberto Sanz

Sonda activa de detección de elementos de red inseguros frente a ataques cuánticos

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **Tutoría del miércoles 14/10/2026**

1. Objetivo de esta tarea

Reproducir en mi propio entorno la sonda activa desarrollada por Roberto Sanz, para entender **qué información obtiene y, sobre todo, qué parte de esa información NO podré obtener en modo pasivo**, que es lo que me corresponde desarrollar en mi TFG.

Mi TFG: desarrollar una sonda en C que capture tráfico de red y analice las conexiones TLS, de forma que se pueda identificar a clientes y servidores que no ofrezcan ni escojan, respectivamente, algoritmos postcuánticos en el establecimiento de sesión.

2. Qué hace la sonda de Roberto

Es una herramienta CLI en Python (`sonda.py`) que, para cada servidor de una lista, lanza `openssl s_client -connect <host> -groups <grupo>` forzando **un único grupo** en el ClientHello. Si el handshake se completa, el servidor soporta ese grupo; si la salida contiene `Cipher is (NONE)`, no lo soporta. Paraleliza con `ThreadPoolExecutor` (≈1000 webs/minuto con 500 hilos).

Punto clave: la sonda no parsea ningún byte de TLS. Toda la criptografía la hace OpenSSL por debajo; el script solo orquesta procesos y busca cadenas de texto en su salida. Por eso puede permitirse estar escrito en Python.

Modos y flags relevantes

Opción	Qué hace
`--host / --file`	Analiza un dominio concreto o una lista desde fichero
`--reduced`	Prueba solo X25519MLKEM768 y SecP256r1MLKEM768 (los dos únicos en producción)
`--re / --rev`	Reintenta los fallos añadiendo el prefijo www.
`-t / -w / -r`	Timeout (10 s por defecto), número de hilos y volcado de resultados a CSV
`--cert-check`	Analiza el certificado buscando firmas PQC (ML-DSA, SLH-DSA, Falcon…)

3. Entorno de pruebas montado

Componente	Versión / detalle
Sistema	Ubuntu 26.04 LTS sobre WSL2 (Windows), editado desde VS Code con Remote-WSL
OpenSSL	**3.5.5** — ya incluye ML-KEM nativo, no hace falta liboqs ni oqsprovider
Python	3.14.4
Repositorio	`github.com/rooberpk16/sonda_tfg` (3 ficheros: LICENSE, requirements.txt, sonda.py)
Dependencias reales	**Ninguna externa:** `subprocess, time, argparse, csv, threading, concurrent.futures`

Nota: el requisito de OpenSSL ≥ 3.5 era el único cuello de botella previsible y se resolvió solo al instalar Ubuntu 26.04, que ya lo trae de serie.

4. Pruebas realizadas y resultados

Comando lanzado sobre cuatro dominios elegidos a propósito (dos de la memoria de Roberto, uno español de referencia y el de la propia universidad):

```
python3 sonda.py --host google.com wikipedia.org caixabank.es uam.es -v --reduced --re
```

Dominio	X25519MLKEM768	SecP256r1MLKEM768	Observación
google.com	**Soportado**	No soportado	Coincide con la memoria
wikipedia.org	**Soportado**	No soportado	**En enero de 2026 NO lo soportaba**
caixabank.es	Error (timeout)	Error (timeout)	Solo responde `www.caixabank.es`
uam.es	No soportado	No soportado	Tampoco con prefijo www.

Diagnóstico del caso caixabank.es

Primero se descartó que fuese un problema de timeout: con `-t 30` el dominio sigue dando Error (mientras que `uam.es` sí pasa de Error a "No soportado", confirmando que ahí 10 s era insuficiente). La comprobación directa cierra el diagnóstico:

```
$ timeout 10 openssl s_client -connect caixabank.es:443 </dev/null; echo exit=$?
      exit=124          (la conexion nunca se completa ni falla)

$ getent hosts caixabank.es
      217.148.70.211   217.148.70.210   217.148.72.211   217.148.72.210
```

El DNS resuelve a cuatro direcciones, pero el puerto 443 **descarta los paquetes en silencio** (ni acepta ni rechaza: código 124 = lo mata el timeout). No es un servidor lento, es un agujero negro. Existe una clase de servidores que simplemente **no se pueden clasificar**.

5. Hallazgos para la tutoría

Hallazgo	Por qué importa
wikipedia.org ya soporta X25519MLKEM768	En la memoria de Roberto (enero 2026) figuraba como no soportado. En nueve meses ha migrado: confirma empíricamente su conclusión sobre la velocidad de adopción y justifica su línea de trabajo futuro de análisis periódicos.
El resumen colapsa «Error» en «No soportado»	Es el fallo más relevante. Su propio requisito RF-1.2 dice que ante un timeout «no se puede determinar», pero el resumen final imprime "Not supported". Consecuencia directa: en el barrido de 10.000 dominios, cada servidor caído, lento o inalcanzable engrosó la cifra de servidores sin PQC, **inflando potencialmente sus porcentajes**.
Timeout por defecto de 10 s demasiado agresivo	Verificado con uam.es: con 10 s devuelve Error y con 30 s resuelve correctamente. La propia memoria recomienda 20–30 s, pero el valor por defecto del código sigue siendo 10.

Hallazgo	Por qué importa
requirements.txt declara dependencias que el código no usa	Lista `oqs`, `pyasn1` y `pyasn1_alt_modules`, pero el script solo importa biblioteca estándar. Sugiere que el repo evolucionó después de la memoria o que quedaron restos de pruebas (pyasn1 apuntaría a un parseo real de ASN.1 que la memoria no describe).
uam.es no soporta ningún grupo postcuántico	La web de la propia universidad no está preparada. Anécdota útil y posible caso de estudio cercano.

6. Qué me llevo para mi sonda pasiva

6.1. La diferencia esencial: control frente a observación

En la sonda activa **yo elijo qué grupos ofrecer**. Por eso puedo afirmar con rotundidad que `uam.es` no soporta X25519MLKEM768: se le ha ofrecido ese grupo y solo ese, y lo ha rechazado.

En pasivo no tengo ese control. Observaré el handshake que toque, con los grupos que el cliente de turno haya decidido ofrecer. **Si un servidor escoge X25519 clásico, no puedo concluir que no soporte PQC**: puede que el cliente simplemente no se lo ofreciera. Esta es la limitación estructural del modo pasivo y hay que resolverla de forma explícita.

6.2. Decisión de diseño que se deriva

- **«Indeterminado» tiene que ser un estado de primera clase**, nunca un alias de «inseguro». Es exactamente el error que comete el resumen de Roberto, y en pasivo lo necesitaré aún más que él, porque tendré dos motivos distintos para no poder concluir: que no vea el handshake completo, y que el cliente nunca ofreciera PQC.
- **Tres opciones para clasificar servidores** (a plantear a Jorge): (a) clasificar solo sobre conexiones informativas, descartando aquellas en que el ClientHello no ofrecía PQC; (b) acumular estado por servidor a lo largo del tiempo, de forma que un servidor al que sí se le ofreció PQC y aun así eligió clásico sea inseguro con certeza; (c) añadir el tercer estado «indeterminado» junto a seguro / inseguro / potencialmente inseguro.
- **El estado «potencialmente inseguro» de la patente es inobservable en una sola conexión**: en cada handshake el servidor elige una cosa y punto. Solo se puede inferir agregando muchas conexiones del mismo servidor.

6.3. Lo que la sonda pasiva aporta frente a la activa

- **Ve los clientes**, que el modo activo no puede observar en absoluto. Es la aportación principal de mi TFG.
- **No es intrusiva**: no abre ninguna conexión. La activa abre una por servidor y por grupo.
- **Dice qué se negocia realmente en la red**, no qué se podría negociar.

7. Siguientes pasos pendientes

Tarea	Estado
Ver y probar el TFG de Roberto	**Hecho**
Leer la patente para entender el contexto	En curso

Tarea	Estado
Repasar IP y TCP (Kurose) y TLS 1.3 en detalle: ClientHello y ServerHello	Pendiente
Extensión key_share (51) y supported_groups (10): grupos PQC e híbridos	Pendiente
Mirar Zeek — ahora como referencia de diseño y herramienta de validación	Pendiente
Ver el TFG de Celia Pascual como base de sonda en C con libpcap	Pendiente
Repasar la práctica de ARP de AR2 (libpcap)	Pendiente
Buscar en GitHub implementaciones de analizadores TLS con libpcap	Pendiente
Decidir libpcap vs Scapy	**Cerrado: C + libpcap** (el enunciado fija el lenguaje)